



King Abdulaziz University
Faculty of Computing & Information Technology
Computer Science Department

Final Exam (Wednesday, April 24th, 2019)
Course: CPCS223
(Analysis and Design of Algorithm)
Term: 2nd Semester (2019), Spring 2019
Time: 120 minutes

Student ID: _____ Section: _____

SOLUTION

Student Name: _____

SO	Questions
a	1
b	2, 3, 4, 5, 6
c	7

Instructors:

- ☐ Dr. Maher Ali Mohammad
- ☐ Dr. Muhammad Umair

Multiple Choice Questions

Question No. 1

(10 Marks)

Q.1

How “Time” and “Space” are generally related to each other?

- a) Directly Proportional
- b) **Inversely Proportional**
- c) Both ways
- d) None

Q.2

What is the value of “q” calculated by $\lfloor (p + r)/2 \rfloor$, if p = 5 and r = 14

- a) **9**
- b) 10
- c) 9.5
- d) None

Q.3

What will be the value of $\sum_{i=0}^{\infty} x^i$ if $x < 1$

- a) **$\frac{1}{1-x}$**
- b) $\frac{1}{1+x}$
- c) $\frac{1}{x-1}$
- d) None

Q.4

Let $X = \lfloor n/4 \rfloor$ and $Y = n/4$, which of the following option is correct, if n is odd?

- a) $X = Y$
- b) $X > Y$
- c) **$X < Y$**
- d) None

Q.5

According to the definition, which condition(s) is / are correct for Theta-Notation (Θ)?

- a) **$f(n) > 0$**
- b) c_1 and c_2 must be positive integers
- c) n can be negative
- d) It describes Upper Bound

Q.6

In which sorting technique at every step each element is placed in its proper position?

- a) Bubble sort
- b) **Quick sort**
- c) Merge sort
- d) Heap sort

Q.7

The running time for build-heap is:

- a) $O(1)$;
- b) $O(\lg n)$;
- c) $O(n \lg n)$;
- d) **$O(n)$;**

Q.8

One of the following is not an elementary operation for Gaussian elimination:

- a) Replacing an equation with the difference of itself and some multiple of another
- b) Replacing an equation by another**
- c) Exchanging two equations
- d) Replacing an equation with the sum of itself and some multiple of another
- e) Replace an equation with its nonzero multiple

Q.9

Each item in priority queue has a value denoting its

- a) Address
- b) Relative Priority**
- c) Search key
- d) Numeric Weight
- e) None of these

Q.10

Which of following algorithm scans the list by swapping the entries whenever pair of adjacent keys are out of desired order?

- a) Insertion Sort
- b) Shell Sort
- c) Quick Sort
- d) Bubble Sort**

HVCLO – 05, SO - b

(Problem Solving)

Question No. 2

(15 Marks)

For the given data set, how the arrays A, count and S will be updated during execution of the given algorithm of “Comparison Counting Sort”? You are required to reflect all changes to these arrays for each iteration.

ALGORITHM *ComparisonCountingSort*($A[0..n - 1]$)
 //Sorts an array by comparison counting
 //Input: An array $A[0..n - 1]$ of orderable elements
 //Output: Array $S[0..n - 1]$ of A 's elements sorted in nondecreasing order
for $i \leftarrow 0$ **to** $n - 1$ **do** $Count[i] \leftarrow 0$
for $i \leftarrow 0$ **to** $n - 2$ **do**
 for $j \leftarrow i + 1$ **to** $n - 1$ **do**
 if $A[i] < A[j]$
 $Count[j] \leftarrow Count[j] + 1$
 else $Count[i] \leftarrow Count[i] + 1$
for $i \leftarrow 0$ **to** $n - 1$ **do** $S[Count[i]] \leftarrow A[i]$
return S

A

0	1	2	3	4	5	6	7	8	9
49	68	05	61	35	57	6	44	99	91

Count

Iteration	0	1	2	3	4	5	6	7	8	9
1	4	1	0	1	0	1	0	0	1	1
2	4	7	0	1	0	1	0	0	2	2
3	4	7	0	2	1	2	1	1	3	3
4	4	7	0	6	1	2	1	1	4	4
5	4	7	0	6	2	3	1	2	5	5
6	4	7	0	6	2	5	1	2	6	6
7	4	7	0	6	2	5	1	3	7	7
8	4	7	0	6	2	5	1	3	8	8
9	4	7	0	6	2	5	1	3	9	8
10	4	7	0	6	2	5	1	3	9	8

S

0	1	2	3	4	5	6	7	8	9
5	6	35	44	49	57	61	68	91	99

Question No. 3**(15 Marks)**

For each of the following lists, construct an AVL tree by inserting their elements successively, starting with the empty tree.

a. A, B, C, D, E, F

b. F, E, D, C, B, A

c. C, D, E, F, A, B

SOLUTION:

HVCLO – 08, SO - b

(Algorithm Comparison)

Question No. 4

PART-A

(20 Marks)

For the given data set, how the arrays A, D and S will be updated during execution of the given algorithm of “Distribution Counting Sort”? You are required to reflect all changes to these arrays, for each iteration.

ALGORITHM *DistributionCountingSort*($A[0..n - 1]$, l , u)

Input: An array $A[0..n - 1]$

Output: Array $S[0..n - 1]$

for $j \leftarrow 0$ to $u - l$ do $D[j] \leftarrow 0$

for $i \leftarrow 0$ to $n - 1$ do $D[A[i] - l] \leftarrow D[A[i] - l] + 1$

for $j \leftarrow 1$ to $u - l$ do $D[j] \leftarrow D[j - 1] + D[j]$

for $i \leftarrow n - 1$ downto 0 do

$j \leftarrow A[i] - l$

$S[D[j] - 1] \leftarrow A[i]$

$D[j] \leftarrow D[j] - 1$

return S

A

0	1	2	3	4	5	6
5	3	5	3	4	3	4

D

(03 pts)

	0	1	2
Initial Values	0	0	0
Frequencies	3	2	2
Distribution	3	5	7

14 pts	D			S						
Iteration	0	1	2	0	1	2	3	4	5	6
1	3	4	7					4		
2	2	4	7			3		4		
3	2	3	7			3	4	4		
4	1	3	7		3	3	4	4		
5	1	3	6		3	3	4	4		5
6	0	3	6	3	3	3	4	4		5
7	0	3	5	3	3	3	4	4	5	5

S

(03 pts)

0	1	2	3	4	5	6
3	3	3	4	4	5	5

Compare the following two algorithm in terms of their performance (Complexity).

ALGORITHM *DistributionCountingSort*($A[0..n-1]$, l , u)

Input: An array $A[0..n-1]$

Output: Array $S[0..n-1]$

for $j \leftarrow 0$ to $u-l$ do $D[j] \leftarrow 0$

for $i \leftarrow 0$ to $n-1$ do $D[A[i]-l] \leftarrow D[A[i]-l] + 1$

for $j \leftarrow 1$ to $u-l$ do $D[j] \leftarrow D[j-1] + D[j]$

for $i \leftarrow n-1$ downto 0 do

$j \leftarrow A[i]-l$

$S[D[j]-1] \leftarrow A[i]$

$D[j] \leftarrow D[j]-1$

return S

Solution:

$\Theta(n)$

ALGORITHM *ComparisonCountingSort*($A[0..n-1]$)

//Sorts an array by comparison counting

//Input: An array $A[0..n-1]$ of orderable elements

//Output: Array $S[0..n-1]$ of A 's elements sorted in nondecreasing order

for $i \leftarrow 0$ to $n-1$ do $Count[i] \leftarrow 0$

for $i \leftarrow 0$ to $n-2$ do

 for $j \leftarrow i+1$ to $n-1$ do

 if $A[i] < A[j]$

$Count[j] \leftarrow Count[j] + 1$

 else $Count[i] \leftarrow Count[i] + 1$

for $i \leftarrow 0$ to $n-1$ do $S[Count[i]] \leftarrow A[i]$

return S

$$C(n) = \sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1$$

$$= \sum_{i=0}^{n-2} [(n-1) - (i+1) + 1]$$

$$= \sum_{i=0}^{n-2} (n-1-i) = \frac{n(n-1)}{2}$$

$$= \Theta(n^2)$$

Solution:

Find the efficiency of the given algorithm in terms of its basic operation(s). You are required to show all steps of calculating the efficiency.

Writing the correct result only will give you 2 points

ALGORITHM *BetterForwardElimination*($A[1..n, 1..n]$, $b[1..n]$)

```

for  $i \leftarrow 1$  to  $n$  do  $A[i, n+1] \leftarrow b[i]$ 
for  $i \leftarrow 1$  to  $n-1$  do
     $pivotrow \leftarrow i$ 
    for  $j \leftarrow i+1$  to  $n$  do
        if  $|A[j, i]| > |A[pivotrow, i]|$   $pivotrow \leftarrow j$ 
    for  $k \leftarrow i$  to  $n+1$  do
         $swap(A[i, k], A[pivotrow, k])$ 
    for  $j \leftarrow i+1$  to  $n$  do
         $temp \leftarrow A[j, i] / A[i, i]$ 
        for  $k \leftarrow i$  to  $n+1$  do
             $A[j, k] \leftarrow A[j, k] - A[i, k] * temp$ 

```

Solution:

$$\begin{aligned}
 C(n) &= \sum_{i=1}^{n-1} \sum_{j=i+1}^n \sum_{k=i}^{n+1} 1 \\
 &= \sum_{i=1}^{n-1} \sum_{j=i+1}^n (n+1-i+1) \\
 &= \sum_{i=1}^{n-1} \sum_{j=i+1}^n (n+2-i) \\
 &= \sum_{i=1}^{n-1} (n+2-i)(n-(i+1)+1) \\
 &= \sum_{i=1}^{n-1} (n+2-i)(n-i) \\
 &= \sum_{j=1}^{n-1} (j+2)j \\
 &= \sum_{j=1}^{n-1} j^2 + \sum_{j=1}^{n-1} 2j
 \end{aligned}$$

$$\begin{aligned}
 C(n) &= \sum_{j=1}^{n-1} j^2 + \sum_{j=1}^{n-1} 2j \\
 &= \frac{(n-1)n(2n-1)}{6} + 2 \frac{(n-1)n}{2} \\
 &= \frac{n(n-1)(2n+5)}{6} \\
 &\approx \frac{1}{3}n^3 \in \Theta(n^3)
 \end{aligned}$$

Find the efficiency of the given algorithm in terms of its basic operation(s). You are required to show all steps of calculating the efficiency.

Writing the correct result only will give you 2 points

ALGORITHM *LeftRightBinaryExponentiation(a, b(n))*

product $\leftarrow a$

for $i \leftarrow I - 1$ **downto** 0 **do**

product $\leftarrow$ *product* * *product*

if $b_i = 1$ *product* $\leftarrow$ *product* * *a*

return *product*

Solution:

$$= \begin{cases} (a^p)^2 & \text{if } b_i = 0, \\ (a^p)^2 \cdot a & \text{if } b_i = 1. \end{cases}$$

$$M(n) = 2(b - 1)$$

$$M(n) = (b - 1)$$

$$(b - 1) \leq M(n) \leq 2(b - 1)$$

$$b - 1 = \lfloor \log_2 n \rfloor$$

The **Dutch national flag problem** is to rearrange an array of characters R, W, and B (red, white, and blue are the colors of the Dutch national flag) so that all the R's come first, the W's come next, and the B's come last. **Design** a linear in-place algorithm for this problem.

Solution:

```
Algorithm DutchFlag( $A[0..n - 1]$ )
//Sorts an array with values in a three-element set
//Input: An array  $A[0..n - 1]$  of characters from {'R', 'W', 'B'}
//Output: Array  $A[0..n - 1]$  in which all its  $R$  elements precede
//      all its  $W$  elements that precede all its  $B$  elements
 $r \leftarrow 0$ ;  $w \leftarrow 0$ ;  $b \leftarrow n - 1$ 
while  $w \leq b$  do
    if  $A[w] = 'R'$ 
        swap( $A[r], A[w]$ );  $r \leftarrow r + 1$ ;  $w \leftarrow w + 1$ 
    else if  $A[w] = 'W'$ 
         $w \leftarrow w + 1$ 
    else //  $A[w] = 'B'$ 
        swap( $A[w], A[b]$ );  $b \leftarrow b - 1$ 
```

Extra Page- If you like any work on this page graded, please label your work clearly.